CN-Stats-Lec02-Types of Data and Data Collection

Types of Data and Data Collection

Session Objectives:

- Get familiar with Python libraries (SciPy, Scikit-learn, StatsModels).
- Identify and categorize different types of data.
- Distinguish between Nominal and Ordinal data.
- Explain Interval and Ratio data.
- Differentiate between Discrete and Continuous data.
- Understand data collection methods & sampling techniques.

Shifting and Scaling

x = 10 20 30 40 50 60 70 80 90 100 110

Q1 Q2 Q3

Mean - 57.27
Median - 60
Range - 100
IQR - Q3-Q1 = 90-30 = 60

x shifts 50

60 70 80 90 100 110 120 130 140 150 160

Q1 Q2 Q3

Mean - 110
Median - 110
Range - 100
IQR - 60

No Change on Shifting

x scale 2

x = 20 40 60 80 100 120 140 160 180 200 220

Q1 Q2 Q3

Mean - 120
Median - 120
Range - 200
IQR - 120

X2 [Original List]

Mean: The Centre of Our Data

Population Mean (μ)

$$\mu = \frac{\sum_{i=1}^{N} x_i}{N}$$

When we have data from the **entire population**, we use capital N and the Greek letter mu (μ) to represent the mean.

Sample Mean (x̄)

$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}$$

When working with a **sample**, we use lowercase n and x-bar (x̄) to represent the mean. This is pronounced "x-bar".

Population vs Sample Variance Formulas

Population Variance (σ^2)

$$\sigma^2 = \frac{\sum_{i=1}^{N} (x_i - \mu)^2}{N}$$

We divide by **N** when we have the complete population. The symbol σ^2 is pronounced "sigma squared".

Sample Variance (s^2)

$$s^2 = \frac{\sum_{i=1}^{n} (x_i - \bar{x})^2}{n - 1}$$

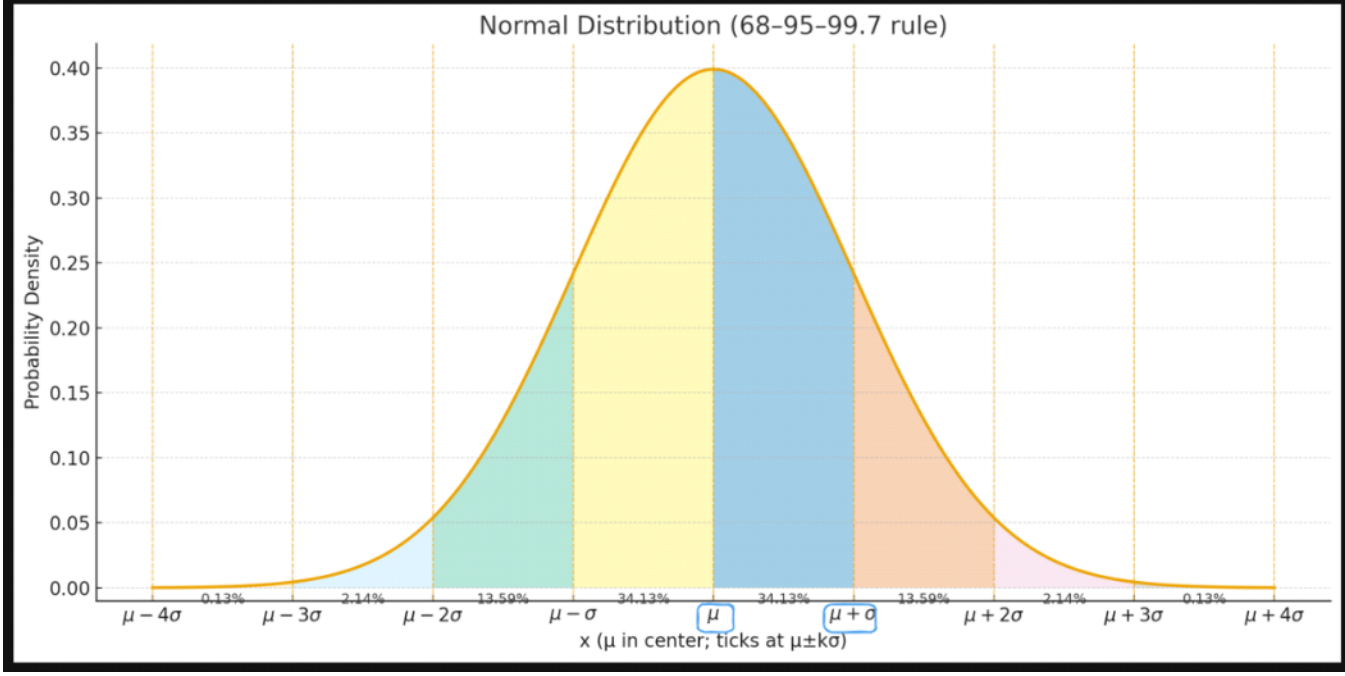
We divide by **n-1** for samples to get unbiased variance. This gives more accurate estimate.

Population Standard Deviation (σ)

$$\sigma = \sqrt{\sigma^2} = \sqrt{\frac{\sum_{i=1}^{N} (x_i - \mu)^2}{N}}$$

Sample Standard Deviation (s)

$$s = \sqrt{s^2} = \sqrt{\frac{\sum_{i=1}^{n} (x_i - \bar{x})^2}{n - 1}}$$



```python
# Scipy
import numpy as np
from scipy import stats
```

```python
dir(stats)
```

```
 'stats',
 'studentized_range',
 't',
 'test',
 'theilslopes',
 'tiecorrect',
 'tmax',
 'tmean',
 'tmin',
 'trapezoid',
 'trapz',
 'triang',
 'trim1',
 'trim_mean',
 'trimboth',
 'truncexpon',
 'truncnorm',
```

Hypothesis Testing
H0: Null Hypothesis &
H1: Alternative Hypothesis

```python
# Generate Data
np.random.seed(11)
data = np.random.normal(loc=50,scale=10,size=1000)
data
```

```
array([67.49454741, 47.13927003, 45.15434868, 23.46681441, 49.91715371,
       46.80368636, 44.63370638, 53.15402668, 54.21050716, 39.3439702 ,
       41.1376033 , 45.24266507, 56.8968231 , 55.61192176, 36.9445149 ,
       38.80524738, 57.36837391, 65.74634073, 49.68924912, 43.16553373,
       60.95629699, 46.90423363, 57.25752225, 65.49071633, 56.30079822,
       50.73493237, 57.32271354, 43.57424608, 48.21906825, 44.26045441,
       47.95624684, 45.135049  , 48.14224676, 46.19463583, 50.88977635,
       50.63671661, 52.96347108, 64.02771119, 34.53137432, 62.95618585,
       47.62749554, 37.67653793, 48.27580233, 50.91838375, 60.67558457,
       39.38365552, 52.17348208, 51.17819504, 33.15889107, 38.14244735,
       56.00102006, 56.95567255, 60.8771086 , 55.33821716, 53.95212013,
       51.22867529, 62.09101642, 41.56933899, 48.58106419, 53.85354138,
       34.22505691, 63.1094364 , 42.07134992, 49.28250588, 71.56674426,
       41.70562748, 44.70627972, 65.61703694, 39.19806175, 45.67942381,
       55.16084037, 54.5539286 , 55.93686203, 53.70506334, 63.45378072,
       60.15942146, 55.95278454, 43.17198384, 42.86440071, 30.91710458,
       56.87312349, 31.76990416, 58.791138  , 68.46364873, 39.38345596,
```

```python
np.random.seed(42)
data = np.random.normal(loc=50,scale=10,size=1000)
data
```

```
array([54.96714153, 48.61735699, 56.47688538, 65.23029856, 47.65846625,
       47.65863043, 65.79212816, 57.67434729, 45.30525614, 55.42560044,
       45.36582307, 45.34270246, 52.41962272, 30.86719755, 32.75082167,
       44.37712471, 39.8716888 , 53.14247333, 40.91975924, 35.87696299,
       64.65648769, 47.742237  , 50.67528205, 35.75251814, 44.55617275,
       51.1092259 , 38.49006423, 53.75698018, 43.9936131 , 47.0830625 ,
       43.98293388, 68.52278185, 49.86502775, 39.42289071, 58.22544912,
       37.7915635 , 52.08863595, 30.40329876, 36.71813951, 51.96861236,
       57.3846658 , 51.71368281, 48.84351718, 46.98896304, 35.2147801 ,
       42.80155792, 45.39361229, 60.57122226, 53.4361829 , 32.36959845,
       53.24083969, 46.1491772 , 43.23078   , 56.11676289, 60.30999522,
       59.31280119, 41.60782477, 46.90787624, 53.31263431, 59.75545127,
       45.20825762, 48.14341023, 38.93665026, 38.03793376, 58.12525822,
       63.56240029, 49.27989878, 60.03532898, 53.61636025, 43.54880245,
       53.61395606, 65.38036566, 49.64173961, 65.64643656, 23.80254896,
       58.21902504, 50.87047068, 47.0099265 , 50.91760777, 30.12431085,
       47.80328112, 53.57112572, 64.77894045, 44.81729782, 41.91506397,
```


```python
np.random.seed(42)
data = np.random.normal(loc=50,scale=10,size=1000)
data
```

```
array([54.96714153, 48.61735699, 56.47688538, 65.23029856, 47.65846625,
       47.65863043, 65.79212816, 57.67434729, 45.30525614, 55.42560044,
       45.36582307, 45.34270246, 52.41962272, 30.86719755, 32.75082167,
       44.37712471, 39.8716888 , 53.14247333, 40.91975924, 35.87696299,
```


`np.random.seed(42)` initializes NumPy's random number generator with a fixed starting point (the integer `42`). This ensures that every time the code is run, the sequence of random numbers generated will be **identical**, enabling **reproducibility**.

```python
type(data)
```

```
numpy.ndarray
```

```python
mean = np.mean(data)
median = np.median(data)
mode = stats.mode(data)
```

```python
print(mean)
print(median)
print(mode)
```

```
50.193320558223256
50.25300612234888
ModeResult(mode=17.58732659930927, count=1)
```

One-Sample t-Test

Used to compare the mean of a single sample ($\bar{x}$) to a known population mean (μ):

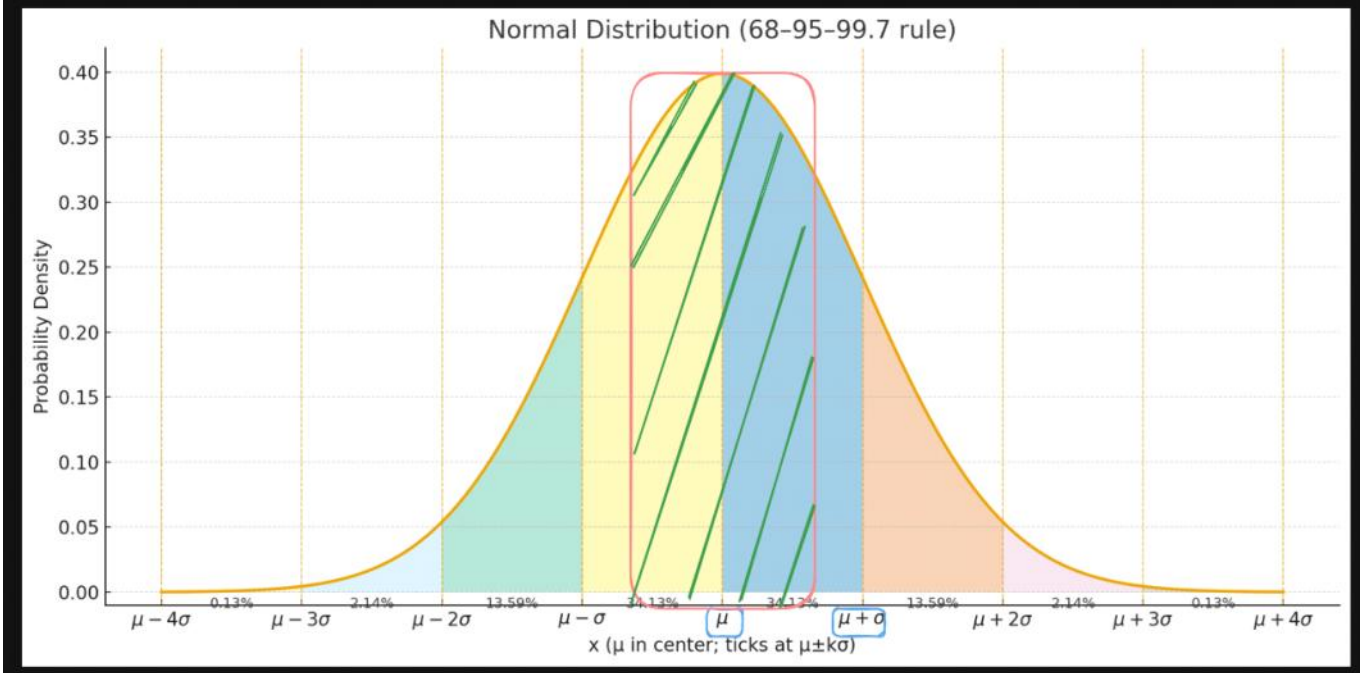
$$t = \frac{\bar{x} - \mu}{s/\sqrt{n}}$$

Where:

- $\bar{x}$ = sample mean
- μ = population mean
- s = sample standard deviation
- n = sample size

```
t_stat , p_value = stats.ttest_1samp(data,50)
print(t_stat)
print(p_value)
```

```
0.6243089585093229
0.5325672436623001
```




```
# Hypothesis Testing [T-Test]
# H0 = Null Hypothesis - 95%
# H1 = Alternative Hypothesis - 5%
# Significance Level alpha (α), - 5% => p_value < 5% (α)
# df = degree of freedom (n-1) = sample size = 50 - 1 = 49
# p_value = probality value to either accept H0 or to reject H0
t_stat , p_value = stats.ttest_1samp(data,50)
print(t_stat)
print(p_value)
```

```
0.6243089585093229
0.5325672436623001
```

```
# p-value = 0.5325 -> 53% to Accept the Null Hypothesis
# H0 : The Sample Mean for the population is = 50.
# H1 : The Sample Mean for the population is != 50.
```

```python
np.random.seed(77)
data = np.random.normal(loc=50,scale=10,size=1000)
data
```

```
array([52.24444852, 56.61531473, 44.20242361, 54.07811324, 54.66479367,
       30.252909  , 52.75548013, 34.24194827, 65.68396907, 42.75204977,
       53.05889717, 61.63942158, 39.5090461 , 40.23421994, 43.60949208,
       66.97429232, 37.13884699, 48.10838418, 49.26087332, 48.39504626,
       48.91206523, 43.03015986, 39.94189192, 46.07454951, 28.62206901,
       67.60930893, 42.25806773, 44.69766066, 42.06483989, 52.91902428,
       37.77994639, 34.30962235, 53.99950159, 49.54355307, 40.84080569,
       61.18986476, 51.9738121 , 52.75060112, 41.2086809 , 39.01760479,
       47.83229509, 39.67935649, 48.3161691 , 55.56938897, 58.12841843,
       48.72887915, 48.47722485, 50.70958812, 50.19954257, 38.79729891,
       32.72796017, 58.8876051 , 45.62196016, 43.58389665, 54.27464041,
       58.30527518, 60.8599978 , 48.62362342, 35.93038724, 61.59348124,
       40.08579994, 51.07351098, 44.55208737, 56.17910288, 79.08049021,
       66.69207801, 54.93889637, 47.27774554, 55.97619852, 48.94091612,
       49.04230633, 64.8675061 , 46.50199174, 61.94764448, 51.906503  ,
       49.05879978, 48.62357051, 51.81989765, 51.63846016, 41.03552282,
       56.04865444, 64.73001086, 59.55840303, 51.50569594, 60.61773916,
```

```python
t_stat , p_value = stats.ttest_1samp(data,50)
print(t_stat)
print(p_value)
```

```
-0.2930989936925345
0.7695073567246077
```

```python
# Scikit-Learn
# Data Normalization
from sklearn.preprocessing import StandardScaler
dir(StandardScaler)
```

```
 '_sklearn_auto_wrap_output_keys',
 '_validate_data',
 '_validate_params',
 'fit',
 'fit_transform',
 'get_feature_names_out',
 'get_metadata_routing',
 'get_params',
 'inverse_transform',
 'partial_fit',
 'set_fit_request',
 'set_inverse_transform_request',
 'set_output',
 'set_params',
 'set_partial_fit_request',
 'set_transform_request',
 'transform']
```

```python
data = np.random.normal(loc=50,scale=10,size=1000)
data = data.reshape(-1,1)
data
```

```
array([[45.11760225],
       [61.99530595],
       [46.01307735],
       [42.95783869],
       [36.73626502],
       [40.55359016],
       [49.18979998],
       [45.81156082],
       [49.44285129],
       [53.4340044 ],
       [60.48563987],
       [39.08963038],
       [40.39249543],
       [38.08978688],
       [48.64047701],
       [65.64711013],
       [47.9116196 ],
```

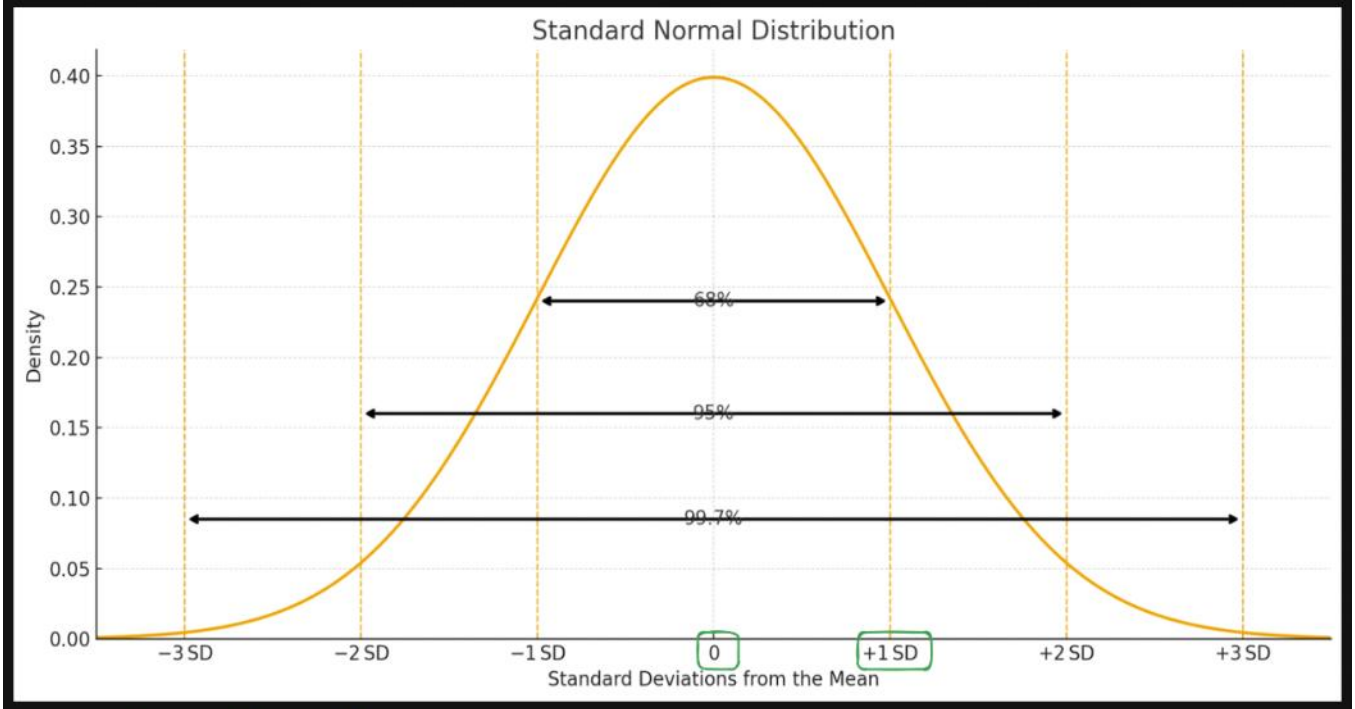
```python
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)
scaled_data
```

```
array([[-4.97838378e-01],
       [ 1.18701738e+00],
       [-4.08445508e-01],
       [-7.13441764e-01],
       [-1.33452473e+00],
       [-9.53451427e-01],
       [-9.13218344e-02],
       [-4.28562361e-01],
       [-6.60604035e-02],
       [ 3.32365675e-01],
       [ 1.03631147e+00],
       [-1.09959459e+00],
       [-9.69533080e-01],
       [-1.19940628e+00],
       [-1.46159269e-01],
       [ 1.55156716e+00],
       [-2.18919143e-01],
```

```python
mean = np.mean(scaled_data)
median = np.median(scaled_data)
std = np.std(scaled_data)

print(mean)
print(median)
print(std)
```

```
-1.0658141036401503e-17
0.019958232865391626
1.0
```



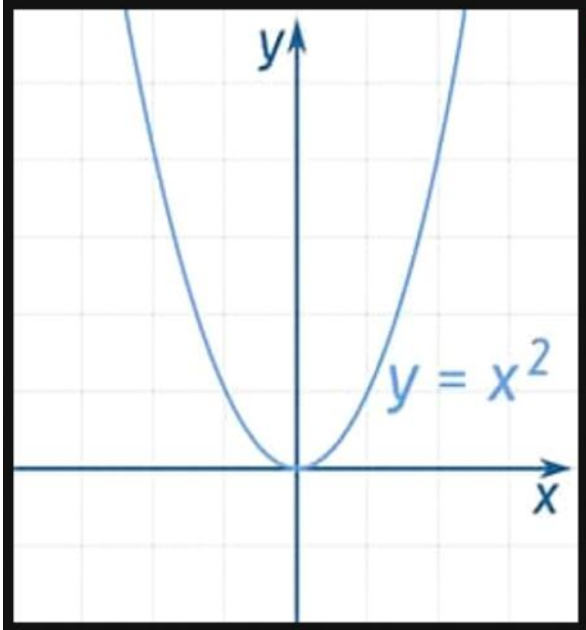

```
mean = 0 , median = 0
std = 1
```

```python
mean = np.mean(scaled_data)
median = np.median(scaled_data)
std = np.std(scaled_data)

print(mean)
print(median)
print(std)
```

```
-1.0658141036401503e-17
0.019958232865391626
1.0
```

PolynomialFeatures



X^2 + 2x + 9 = 0

```python
# Polynomial Features
from sklearn.preprocessing import PolynomialFeatures
# X = np.random.rand(shape)
X = np.random.rand(1000,1) * 10 # range (0,10)
poly = PolynomialFeatures(degree = 2)
X_poly = poly.fit_transform(X)
X_poly
```

```
array([[ 1.        ,  1.08905192,  1.18603408],
       [ 1.        ,  5.74000343, 32.9476394 ],
       [ 1.        ,  8.77327948, 76.97043285],
       ...,
       [ 1.        ,  0.93775867,  0.87939132],
       [ 1.        ,  6.49339568, 42.16418744],
       [ 1.        ,  2.35154546,  5.52976605]])
```

```
X
```

```
array([[1.08905192e+00],
       [5.74000343e+00],
       [8.77327948e+00],
       [3.22779793e+00],
       [6.32219841e-01],
       [6.40348730e+00],
       [8.30691272e+00],
       [2.00159399e+00],
       [6.61829642e+00],
       [6.01367461e+00],
       [8.17284854e-01],
       [2.29707590e+00],
       [6.23525406e+00],
       [9.41108367e+00],
       [7.47839522e+00],
       [4.81386622e+00],
       [2.05659509e+00],
```

```
mean = np.mean(X_poly)
median = np.median(X_poly)
std = np.std(X_poly)
print(mean)
print(median)
print(std)
```

```
13.044651091712142
3.3515809003514176
22.102341307163584
```

```
# Polynomial Features [degree = 3]
from sklearn.preprocessing import PolynomialFeatures
# X = np.random.rand(shape)
X = np.random.rand(1000,1) * 10 # range (0,10)
poly = PolynomialFeatures(degree = 3)
X_poly = poly.fit_transform(X)
X_poly
```

```
array([[1.00000000e+00, 9.61087594e+00, 9.23689364e+01, 8.87746388e+02],
       [1.00000000e+00, 1.45902699e-01, 2.12875975e-02, 3.10591792e-03],
       [1.00000000e+00, 9.67301669e+00, 9.35672518e+01, 9.05077588e+02],
       ...,
       [1.00000000e+00, 2.68416700e-01, 7.20475251e-02, 1.93387590e-02],
       [1.00000000e+00, 2.27904414e+00, 5.19404220e+00, 1.18374515e+01],
       [1.00000000e+00, 5.25202331e+00, 2.75837488e+01, 1.44870492e+02]])
```

```
mean = np.mean(X_poly)
median = np.median(X_poly)
std = np.std(X_poly)
print(mean)
print(median)
print(std)
```

```
67.65887452819575
5.3855629342957965
168.86457585067956
```

```python
# StatsModel
import pandas as pd
import statsmodels.api as sm
dir(sm.stats)
```

```
['AnovaRM',
 'CompareMeans',
 'DescrStatsW',
 'Describe',
 'FTestAnovaPower',
 'FTestPower',
 'FactoredPSDMatrix',
 'GofChisquarePower',
 'Mediation',
 'NormalIndPower',
 'NullDistribution',
 'OaxacaBlinder',
 'RegressionFDR',
 'Runs',
 'SquareTable',
 'StratifiedTable',
 'TTestIndPower',
```

```python
data = np.random.normal(loc=50,scale=10,size=1000)
df = pd.DataFrame(data, columns = ['Value'])
df
```

	Value
0	56.597539
1	47.913522
2	35.476874
3	48.151112
4	41.503609
...	...
995	42.922185
996	34.728729
997	40.790370
998	63.534198
999	27.520406

1000 rows × 1 columns

```python
stats_model = sm.stats.DescrStatsW(df['Value'])
stats_model
```

```
<statsmodels.stats.weightstats.DescrStatsW at 0x1ad242aabd0>
```

```python
mean = stats_model.mean
median = df['Value'].median()
print(mean)
print(median)
```

```
50.051062143564224
49.75379675853475
```